

Bioinformatics
Module 3 – session 3

Supervised learning

(machine learning supervisionada)

...

(objetivo é ver qual é o workflow do trabalho de machine learning supervisionado)



Summary

- Supervised Machine learning definitions and workflow
 - Supervised ML workflow
 - Error metrics and error estimation
 - Learning bias and overfitting
- Main ML models and training algorithms
 - KNN
 - Trees
 - Functional models: linear/ logistic regression, SVMs
- Definições e fluxo de trabalho supervisionados de aprendizado de máquina
 - Fluxo de trabalho de ML supervisionado
 - Métricas de erro e estimativa de erro
 - Viés de aprendizagem e overfitting
- Principais modelos de ML e algoritmos de treinamento
 - KNN
 - Árvores
 - Modelos funcionais: regressão linear/logística, SVMs

(a diferença é que no supervisionado temos a capacidade de escolher o output que queremos)

Machine Learning Paradigms

Unsupervised

no external indication is provided, and learning is accomplished by discovering regularities/ patterns in the input data (e.g. clustering, dimensionality reduction).

Supervised

a correct answer is provided for each situation; learning is performed from examples composed of an input (typically a vector of values for a set of features) and a desired output.

Semi-supervised

an hybrid of the previous two cases, where only part (typically a smaller part) of the examples have known labels and other data is unlabelled

é fornecida uma resposta correta para cada situação; o aprendizado é realizado a partir de exemplos compostos por uma entrada (normalmente um vetor de valores para um conjunto de características) e uma saída desejada.

(podemos atraves de dados clinicos prever o diagnostico (podemos ter a certeza do diagnostico)
o machine learning precisa de dados onde se tem inputs e outputs)

The learning problem

Suppose a patient goes to the doctor with certain symptoms of disease. The doctor needs to diagnose if the patient has flu or not. She/he will need to observe certain symptoms of the patient:

- has a headache
- what temperature does he/she has
- does it have throat inflammation
- etc

The doctor can compile all this information in a **dataset** so it can be used to build a **ML model**.

All the symptoms are considered as ***input variables*** and the disease outcome is the ***output variable***.

Suponha que um paciente vá ao médico com certos sintomas de doença. O médico precisa diagnosticar se o paciente está gripado ou não. Ela/ele precisará observar alguns sintomas do paciente:

- tem dor de cabeça
- que temperatura ele/ela tem? • ele/ela tem inflamação na garganta
- etc.

O médico pode compilar todas essas informações em um conjunto de dados para que possam ser usadas na construção de um modelo de ML.

Todos os sintomas são considerados variáveis de entrada e o desfecho da doença é a variável de saída.

Datasets for supervised learning

Inputs:				Output	
Features	Headache	Temperature	Age	Throat ache	Flu?
Examples	No	36.6	35	No	No
	No	38.3	40	Yes	No
	Moderate	38.6	86	No	Yes
	Strong	40.0	26	No	Yes
	Strong	38.2	50	No	Yes
	Strong	36.5	70	Yes	No
	Moderate	39.1	65	Yes	Yes
	No	38.3	15	No	No
	(...)				
Feature types	Discrete	Continuous	Continuous	Discrete	Discrete

Classification:
Output feature
is discrete

Regression:
Output feature
is continuous

(cada coluna tem uma feature e ha dados continuos (ou numericos) e discretos)

Classification:
Output feature
is discrete

Regression:
Output feature
is continuous

(se em vez de gripe se quisesse algo numerico (ex: quantos dias demora a ficar bem),era outra abordagem)

(a coluna esta a amarelo porque é um dataset para um problema supervisionado (é a coluna que queremos prever - queremos prever se o paciente tem ou não gripe))

(este algoritmo é diferente e que nos dá como input a entrada do algoritmo e queremos prever com poucos erros mas isto causa alguns problemas pois é necessário assumir mais do que o que se sabe com certeza. Assim, se coisas que não são totalmente verdade.

os modelos de machine learning falham naturalmente)

Supervised learning – models and algorithms

A **model** (MODELO- função que se tem inputs e pode prever se os outputs. o domínio da previsão depende do domínio de output)

- is a function that allows you to obtain **the value of output variables from the values of input variables**;
- generalizes from existing data.

Learning algorithm:

- allows to **obtain a model from a set of training examples** (where inputs and outputs are known);
- perform a process of searching for the best model defined by a given representation structure

Um modelo:

- é uma função que permite obter o valor das variáveis de saída dos valores das variáveis de entrada;
- generaliza a partir de dados existentes.

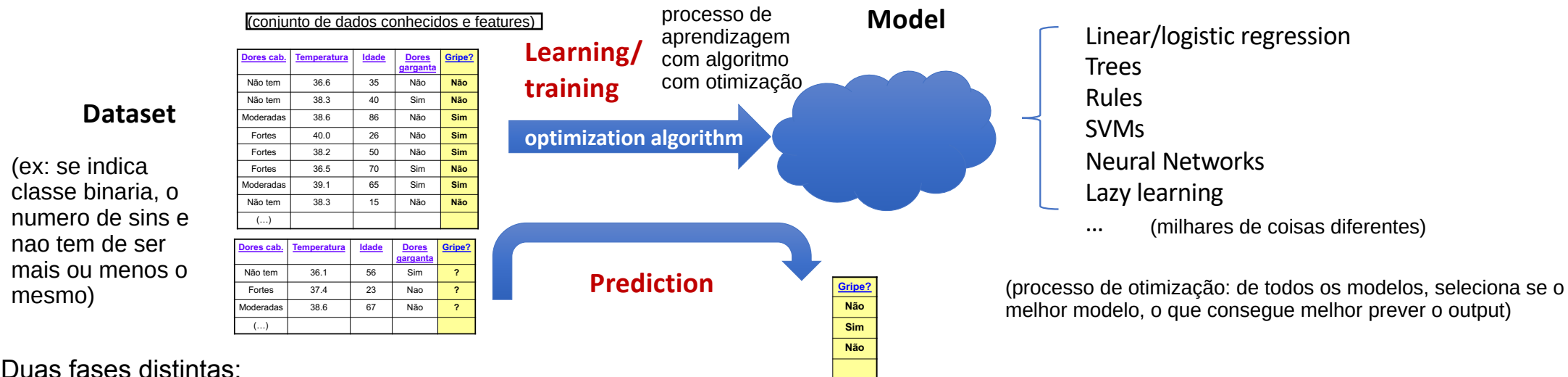
Algoritmo de aprendizagem:

- permite obter um modelo a partir de um conjunto de exemplos de treinamento (onde as entradas e saídas são conhecidas);
- realizar um processo de busca pelo melhor modelo definido por uma determinada estrutura de representação

Supervised learning – workflow

Two distinct phases:

- **Model construction (training)** from the known data (inputs and outputs), given a representation structure, determines the value of model parameters to minimize a cost function – **optimization problem**
- **Prediction** - Application of the model to data where only the value of the input attributes is known and the value of the output attribute is predicted



Duas fases distintas:

- A construção do modelo (treinamento) a partir dos dados conhecidos (entradas e saídas), dada uma estrutura de representação, determina o valor dos parâmetros do modelo para minimizar uma função de custo – problema de otimização
- Predição - Aplicação do modelo a dados onde apenas o valor dos atributos de entrada é conhecido e o valor do atributo de saída é previsto

Model Evaluation: error metrics

The evaluation of the quality of a model for a given task is done by calculating **error metrics** on a given set of examples (not used in the construction of the model)

These depend on the type of problem: **classification** or **regression**

A avaliação da qualidade de um modelo para uma determinada tarefa é feita calculando métricas de erro sobre um determinado conjunto de exemplos (não utilizados na construção do modelo)

Dependem do tipo de problema: classificação ou regressão

Classification: confusion matrix

In classification problems, the confusion matrix is usually calculated, which maps the predicted values (by a model) to the desired values

Desired (real) values	Predicted values	
	Negative	Positive
	Negative	Positive
Negative	TN Negativo	FP Falso positivo
Positive	FN Falso negativo	TP Positivo

(podemos saber quais as metricas que se podem usar. ha casos positivos e negativos)

(é uma matriz)

(na saude é pior o falso negativo mas no spam do mail por exemplo é pior o falso positivo)

Example for two classes

Em problemas de classificação, geralmente é calculada a matriz de confusão, que mapeia os valores previstos (por um modelo) para os valores desejados

Error measures: classification

Accuracy

$$\text{Accuracy} = (\text{TN} + \text{TP}) / (\text{TN} + \text{TP} + \text{FP} + \text{FN}) (\times 100\%)$$

(pode nao ser muito viavel por isso retira se muitas vezes o TN por ex no caso do COVID)

For more than two classes

$$\text{Accuracy} = \text{Number of correctly classified examples} / \text{Total number of dataset examples}$$

Sensitivity (type II error): (aka **recall**) (1-sensitivity = recall = type II error)

$$\text{sensitivity} = \text{TP} / (\text{FN} + \text{TP})$$

$$E(\text{type II}) = \text{FN} / (\text{FN} + \text{TP}) = 1 - \text{sensitivity}$$

Specificity (type I error)

$$\text{specificity} = \text{TN} / (\text{TN} + \text{FP})$$

$$E(\text{type I}) = \text{FP} / (\text{FP} + \text{TN}) = 1 - \text{specificity}$$

Error measures: classification

Positive predictive value (aka **precision**)

$$PPV = TP / (TP + FP)$$

In case there are more than 2 classes, the values are calculated as if there were 2x2 confusion matrixes for each class, where the "negative" values are the elements of all other classes

F-measure

$$F1 = 2 \times (\text{precision} \times \text{recall}) / (\text{precision} + \text{recall})$$

(combinação do precision e recall)

MCC – Matthews Correlation Coefficient

Works best for cases with imbalance between the two classes

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

Caso existam mais de 2 classes, os valores são calculados como se existissem matrizes de confusão 2x2 para cada classe, onde os valores “negativos” são os elementos de todas as outras classes

Neste caso, as medidas são calculadas com base no erro cometido para cada exemplo. Esta é a diferença entre o valor previsto ($\hat{y}$) e o valor real (y)

Error measures - regression

(na regressao as medidas sao mais simples no sentido em que se trabalha com valores numericos)

In this case, measures are calculated on the basis of the error made for each example. This is the difference between the predicted value ($\hat{y}$) and the actual value (y)

For N examples different error metrics can be used:

- Sum of squared errors (SSE)
- Square root of SSE (RMSE)
- Mean of absolute errors (MAE)
- **R^2** Statistic, scaled between 0 and 1 tells how close the data is to the fitted regression line.

$$RMSE = \sqrt{\frac{1}{n} \sum_i^n (y_i - \hat{y}_i)^2}$$

(o r^2 diz da variabilidade dos outputs qual é a percentagem q esta a prever (EX: $r^2=0,8$ significa que consegue prever 80%))

$$R^2 = \frac{\text{Explained Variance}}{\text{Total Variance}}$$

These values can also be standardised so that they can be compared between different problems.

Para N exemplos, diferentes métricas de erro podem ser usadas:

- Soma dos erros quadráticos (SSE)
- Raiz quadrada de SSE (RMSE)
- Média dos erros absolutos (MAE)
- A estatística R^2 , dimensionada entre 0 e 1, informa o quão próximos os dados estão da linha de regressão ajustada.

Esses valores também podem ser padronizados para que possam ser comparados entre diferentes problemas.

Error estimation methods

Objective: evaluate **credibly** the performance of an algorithm and its generalization ability

You can't do this by applying an error measure to the same dataset that gave rise to the model – you need to define a set of **test examples**

Test examples should be taken from the same population **independently** of training examples

Objetivo: avaliar com credibilidade o desempenho de um algoritmo e sua capacidade de generalização

Você não pode fazer isso aplicando uma medida de erro ao mesmo conjunto de dados que deu origem ao modelo – você precisa definir um conjunto de exemplos de teste

Os exemplos de teste devem ser retirados da mesma população, independentemente dos exemplos de treinamento

Holdout

Most popular method: division of the initial set of examples into 2 groups – **train + test**

This method takes the name of **holdout**.

It is simple and fast, but very sensitive to how the sample is divided.

It can be repeated a number of times, to gain statistical significance (**resampling**)



Método mais popular: divisão do conjunto inicial de exemplos em 2 grupos – treino + teste

Este método leva o nome de holdout.

É simples e rápido, mas muito sensível à forma como a amostra é dividida.

Pode ser repetido várias vezes, para obter significância estatística (reamostragem)

Cross validation

(validação cruzada)

- Improvement of the holdout method that allows the use of all available data.
- **K-fold cross validation**
 - Examples are divided into k mutually exclusive subsets (of identical sizes);
 - Algorithm is applied and tested k times;
 - In each iteration i ($i=1, \dots, k$): model created from all examples except those in group i and tested only in the examples in group i;
 - Final error is calculated by averaging the errors in each iteration.

TRAIN	TRAIN	TEST
TRAIN	TEST	TRAIN
TEST	TRAIN	TRAIN

Example
for k=3

(não se testa o
modelo e usam-se
as mesmas bases)

- Melhoria do método holdout que permite a utilização de todos os dados disponíveis.
- Validação cruzada K-fold
 - Os exemplos são divididos em k subconjuntos mutuamente exclusivos (de tamanhos);
 - Algoritmo é aplicado e testado k vezes;
 - Em cada iteração i ($i=1, \dots, k$): modelo criado a partir de todos os exemplos exceto os do grupo i e testado apenas nos exemplos do grupo i;
 - O erro final é calculado pela média dos erros em cada iteração.

Learning bias

(nada nos garante que os datasets são lineares)

Given a set of examples, there are always numerous possible models that explain these examples satisfactorily.

To be able to generalize, we have to impose a "bias" or restrictions on the process of building the model – **learning bias**.

The learning bias can be reflected at several levels, of which the following stand out:

- Representation structure (what is the structure of the model);
- Search process for the best model (what is the algorithm? What is the cost function)

Dado um conjunto de exemplos, há sempre numerosos modelos possíveis que explicam estes exemplos de forma satisfatória.

Para poder generalizar, temos que impor um "viés" ou restrições ao processo de construção do modelo – viés de aprendizagem.

O viés de aprendizagem pode refletir-se em vários níveis, dos quais se destacam:

- Estrutura de representação (qual é a estrutura do modelo);
- Processo de busca pelo melhor modelo (qual é o algoritmo? Qual é a função de custo)

(os modelos devem ser tao complexos quanto necessário, não mais e não menos)

Overfitting

One of the biggest problems in the learning process: the model learns "too well" the examples and **loses ability to generalize**, i.e. the error in the training examples decreases, but in the universe of interest it increases.

How can we avoid it ?

- Having more data;
- Avoiding models that are too complex in relation to the available data;
- Avoiding prolonging the learning process too much ...

In practice, the problem is difficult to solve ...

- If we have simpler models, we can run the risk of under-fitting (models with insufficient complexity)
- Each dataset requires distinct configurations of the algorithms/models

Um dos maiores problemas no processo de aprendizagem: o modelo aprende “muito bem” os exemplos e perde a capacidade de generalizar, ou seja, o erro nos exemplos de treinamento diminui, mas no universo de interesse aumenta.

Como podemos evitá-lo?

- Ter mais dados;
- Evitar modelos demasiado complexos em relação aos dados disponíveis;
- Evitar prolongar muito o processo de aprendizagem...

Na prática, o problema é difícil de resolver...

- Se tivermos modelos mais simples, podemos correr o risco de under-fitting (modelos com complexidade insuficiente)
- Cada conjunto de dados requer configurações distintas dos algoritmos/modelos

(este metodo inclui se dentro do lazy learning porque se recebe os dados e nao se faz nada, apenas se faz comparação.)

K-nearest neighbors (KNN) algorithm

Most popular method of instance-based learning/lazy learning

How it works:

- **training**: store the available exemples;
- when you want to **classify a new example**:
 - calculate the **k most similar examples** to the example to classify (using input features);
 - Calculate the most common class in the k examples – this will be the predicted class
 - For **regression problems**, the output feature values are averaged in the K examples

Need to define a **similarity function** between two examples, desirably normalized (e.g. $[0,1]$) – not always simple !

Método mais popular de aprendizagem baseada em instâncias/aprendizagem preguiçosa

Como funciona:

- treinamento: armazenar os exemplos disponíveis;
- quando você deseja classificar um novo exemplo:
 - calcular os k exemplos mais semelhantes ao exemplo a classificar (utilizando características de entrada);
 - Calcule a classe mais comum nos k exemplos – esta será a classe prevista
 - Para problemas de regressão, a média dos valores dos recursos de saída é calculada nos K exemplos

Necessidade de definir uma função de similaridade entre dois exemplos, desejavelmente normalizada (por exemplo, $[0,1]$) – nem sempre simples!

(probabilidades que relacionam inputs com outputs (ex: qual a probabilidade de ter febre se se tem gripe))

Probabilistic models: Naive-Bayes

Based on the treatment of conditional probabilities: **Bayes theorem**

Core ideas

- calculate the **probabilities** associated to the belonging of an example to each of the possible classes
- use **co-occurrence frequencies** of class values and values for input attributes to estimate probabilities (for numerical attributes a distribution is assumed and parameters are estimated based on the training data)

Assumptions (which rarely occur in reality...)

- All attributes have the same importance
- values for the various attributes occur **independently**

Com base no tratamento de probabilidades condicionais: teorema de Bayes

Ideias centrais

- calcular as probabilidades associadas à pertença de um exemplo a cada uma das classes possíveis
- usar frequências de co-ocorrência de valores de classe e valores para atributos de entrada para estimar probabilidades (para atributos numéricos é assumida uma distribuição e os parâmetros são estimados com base nos dados de treinamento)

Suposições (que raramente ocorrem na realidade...)

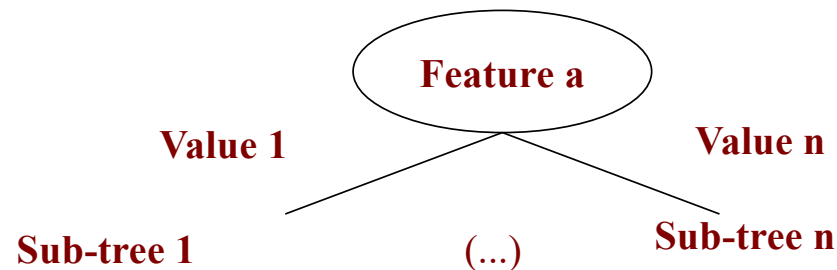
- Todos os atributos têm a mesma importância
- os valores para os vários atributos ocorrem de forma independente

Decision trees: representation structure

N-ary trees where each node specifies a test to the value of a given input feature;

Each **branch** that exits this node corresponds to a possible value for this feature, and gives rise to a subtree of similar structure

In the case of a tree **leaf** this presents a class, that is, a value for the output attribute.



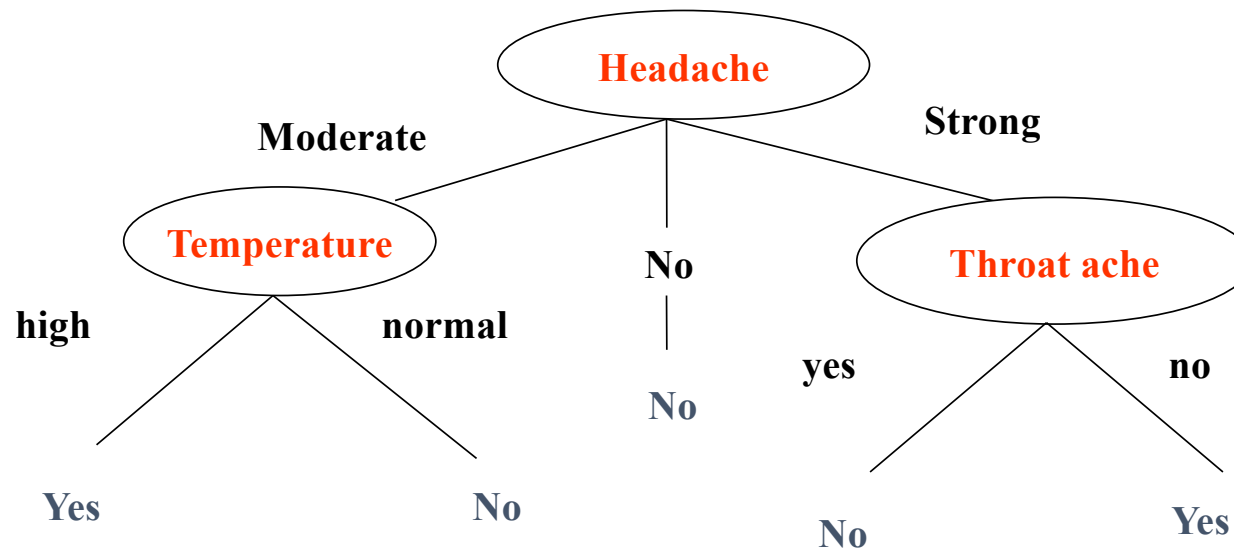
Árvores n-árias onde cada nó especifica um teste para o valor de um determinado recurso de entrada;

Cada ramo que sai deste nó corresponde a um valor possível para este recurso e dá origem a uma sub-árvore de estrutura semelhante

No caso de uma folha de árvore apresenta uma classe, ou seja, um valor para o atributo de saída.

(classificação com base numa árvore
a árvore é constituída de cima para baixo
é um modelo discreto: só tem 5 opções de output)

Decision trees (example)



Notice that you can look at a decision tree classifier, for each class, as a logical OR (for each leaf with that label) of ANDs (joining all the branches passed in the route to reach that leaf)

Observe-se que se pode olhar para um classificador de árvore de decisão, para cada classe, como um OR lógico (para cada folha com aquele rótulo) de ANDs (unindo todos os ramos passados na rota para chegar a essa folha)

Classification of examples

Each example is classified by a recursive process, initiated from the root of the tree by testing the feature defined therein;

According to the value of the feature in the example to classify, the respective branch is followed;

The process is repeated in the respective sub-tree;

The process ends when you reach a leaf of the tree, where there is a class (result of the classification process).

Cada exemplo é classificado por um processo recursivo, iniciado a partir da raiz da árvore, testando o recurso nela definido;

De acordo com o valor do traço do exemplo a classificar, segue-se o respectivo ramo;

O processo é repetido na respectiva subárvore;

O processo termina quando se chega a uma folha da árvore, onde existe uma classe (resultado do processo de classificação).

Decision tree construction

Made starting from the root to the leaves (**top-down**);

It begins with the question, "What is the best feature to test?"

This will be chosen as the one that can “best” classify the examples given- selected based on metrics such as the Gini index or gain (entropy reduction)

For each value of the chosen feature, a branch is created

The subtree on each branch is constructed recursively following the same process (ignoring the features tested above).

Feito a partir da raiz até as folhas (de cima para baixo);

Começa com a pergunta: “Qual é o melhor recurso para testar?”

Este será escolhido como aquele que pode “melhor” classificar os exemplos dados – selecionados com base em métricas como o índice de Gini ou ganho (redução de entropia)

Para cada valor do recurso escolhido, uma ramificação é criada

A subárvore em cada ramo é construída recursivamente seguindo o mesmo processo (ignorando os recursos testados acima).

(as árvores também podem ser aplicadas em regressões)

Regression trees

Tree models can be easily adapted to **regression**: class value (leafs) on the tree becomes a numeric value.

Construction algorithms:

- Algorithm similar to decision trees – top-down strategy
- criterion used to choose the attribute to test on each node
 - base idea will be the same: choose the attribute that best divides the examples in relation to the output feature
 - numerical measure of homogeneity: standard deviation
 - choose the feature that maximizes the **reduction of standard deviation** in the output attribute value

Os modelos de árvore podem ser facilmente adaptados à regressão: o valor da classe (folhas) na árvore torna-se um valor numérico.

Algoritmos de construção:

- Algoritmo semelhante a árvores de decisão – estratégia top-down
- critério utilizado para escolher o atributo a ser testado em cada nó
 - a ideia base será a mesma: escolha o atributo que melhor divide os exemplos em relação ao recurso de saída
 - medida numérica de homogeneidade: desvio padrão
 - escolher o recurso que maximiza a redução do desvio padrão no valor do atributo de saída

Functional models

Represent relationships between **input variables** $X_1, \dots, X_p$ (independent variables), and an **output** variable Y (dependent).

Representam relacionamentos entre variáveis de entrada $X_1, \dots, X_p$ (independentes variáveis) e uma variável de saída Y (dependente).

$$\hat{y} = f(x_1, \dots, x_p; \theta)$$

Model prediction given by:

p – number of inputs

ϑ - parameters of the model

(gradiente é a derivada das funções)

Functional models: parameter estimation

Knowing the structure of the model -> **numerical optimization problem** – Minimizing a cost function (typically considering an error metric)

In some simpler cases (e.g. linear models), analytical methods can be used (e.g **least squares** method).

Gradient descent algorithms are used in many cases where the cost function is differentiable, although they may converge to local minima.

Conhecer a estrutura do modelo -> problema de otimização numérica – Minimizar uma função de custo (normalmente considerando uma métrica de erro)

Em alguns casos mais simples (por exemplo, modelos lineares), podem ser utilizados métodos analíticos (por exemplo, método dos mínimos quadrados).

Algoritmos de gradiente descendente são usados em muitos casos onde a função de custo é diferenciável, embora possam convergir para mínimos locais.

Data preparation: treatment of nominal attributes

In many functional models, all features have to be **numerical** – need to convert nominal to numerical values

Hypothesis 1: divide the numerical range allowed by the number of attribute values: 1 nominal feature = 1 numerical feature

Hypothesis 2: binarize the feature: 1 nominal attribute with M possible values = M numeric attributes with possible values 0 and 1 (*one-hot encoding*)

Em muitos modelos funcionais, todos os recursos devem ser numéricos – é necessário converter valores nominais em numéricos

Hipótese 1: divida a faixa numérica permitida pelo número de valores de atributos: 1 característica nominal = 1 característica numérica

Hipótese 2: binarizar o recurso: 1 atributo nominal com M valores possíveis = M atributos numéricos com valores possíveis 0 e 1 (codificação one-hot)

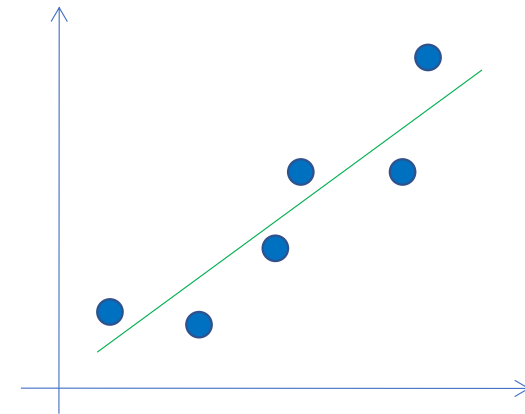
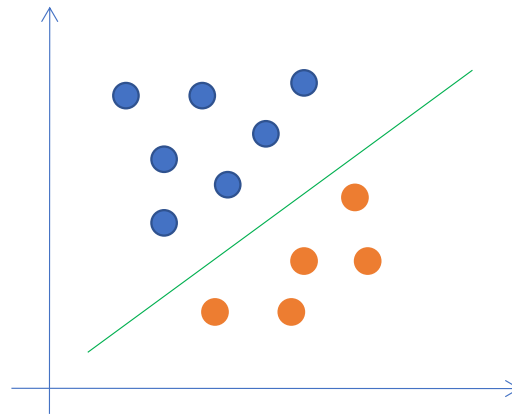
Linear models

Attractive, given the simplicity of calculation and analysis

Can be used for **classification** (separation between classes) or **regression**

Atraente, dada a simplicidade de cálculo e análise

Pode ser usado para classificação (separação entre classes) ou regressão



Linear regression models

$$\hat{y}^{(i)} = h_{\theta}(x^{(i)}) = \theta_0 + \sum_{j=1}^n \theta_j x_j^{(i)}$$

(modelo de otimização
numérica no caso do
modelo linear)

θ_i – model parameters

Linear regression models

Error function (cost): mean squared error - MSE

$$J_{\theta} = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Value predicted by the model: $\hat{y}^{(i)}$ Real value



J is a function of the model parameters $\theta_1, \dots, \theta_n$

Objective: identify the parameters of the model in order to
minimize the value of J

Objetivo: identificar os parâmetros do modelo a fim de minimizar o valor de J

Logistic regression

Discrete dependent variable: problem of **classification**

Logistic regression: uses regression models for binary classification interpreting model output to extract a class

Variável dependente discreta: problema de classificação

Regressão logística: utiliza modelos de regressão para interpretação de classificação binária
saída do modelo para extrair uma classe

$$h_{\theta}(x) = g(\theta^T x) \quad 0 \leq h_{\theta}(x) \leq 1$$

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

$$\frac{1}{1 + e^{-z}}$$

Sigmoid function

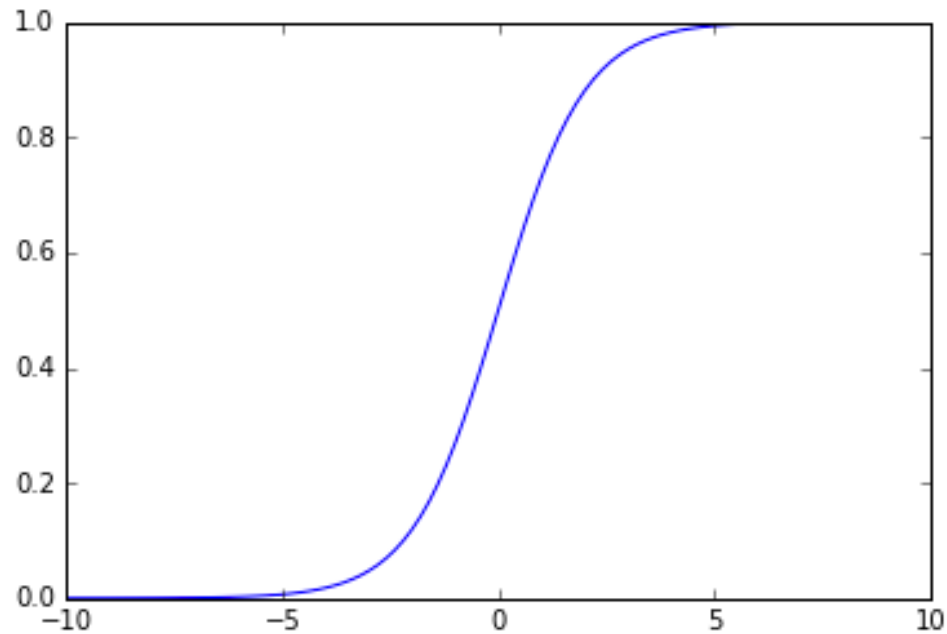
Model is given by applying the sigmoid to the linear regression function

Interpretation: h estimates the **probability** of y (output) being equal to 1 to the example x

O modelo é dado aplicando o sigmóide à função de regressão linear

Interpretação: h estima a probabilidade de y (saída) ser igual a 1 para o exemplo x

Sigmoid function



$z = 0 \Rightarrow \text{sigmoid}(z) = 0.5$

$z \ll 0 \Rightarrow \text{sigmoid}(z) \rightarrow 0$

$z \gg 0 \Rightarrow \text{sigmoid}(z) \rightarrow 1$

Logistic regression: objective function

Error function (for each example x):

$$\begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

Let $y = 1$:
Seja $y = 1$:
Se a previsão estiver correta: o erro é zero
Caso contrário, à medida que a previsão se aproxima de 0, o erro tende ao infinito

If the prediction is correct: error is zero

Otherwise, as the prediction gets closer to 0, error tends to infinity

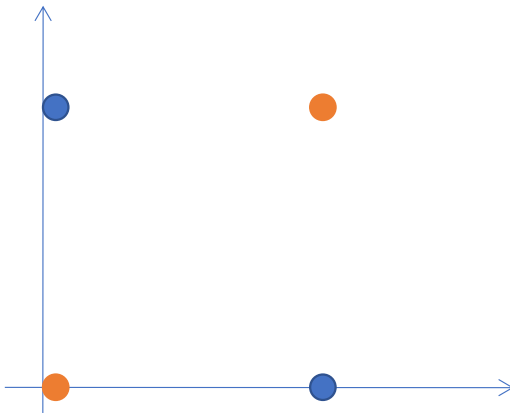
Let $y = 0$:

If prediction is correct: error is zero

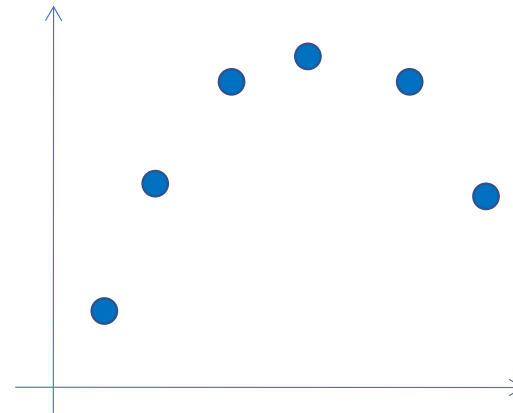
Otherwise, as the prediction gets closer to 1, error tends to infinity

Seja $y = 0$:
Se a previsão estiver correta: o erro é zero
Caso contrário, à medida que a previsão se aproxima de 1, o erro tende ao infinito

Non linear problems



Classification
(e.g. XOR)



Regression

(sao um modelo muito utilizado e bom em termos de desempenho e tem 2 grandes ideias:

1) posso usar a discriminação de margem maxima (slide seguinte)

2) como se quer modelos nao lineares vai se transformar as variaveis permitindo trabalhar no espeço linear)

Support Vector Machines - SVMs

SVMs were developed by Vapnik in the early 1990s

They have been gaining popularity as classification and regression tools, due to a number of theoretical advantages and empirical results

They are based on 2 ideas:

1 - use only the **relevant examples** in the training set (support vectors) and

2 - use a nonlinear transformation applied to the inputs for a linear space of linear via a function of **kernel**

SVMs foram desenvolvidos por Vapnik no início de 1990

Eles vêm ganhando popularidade como classificação e regressão ferramentas, devido a uma série de vantagens teóricas e resultados empíricos

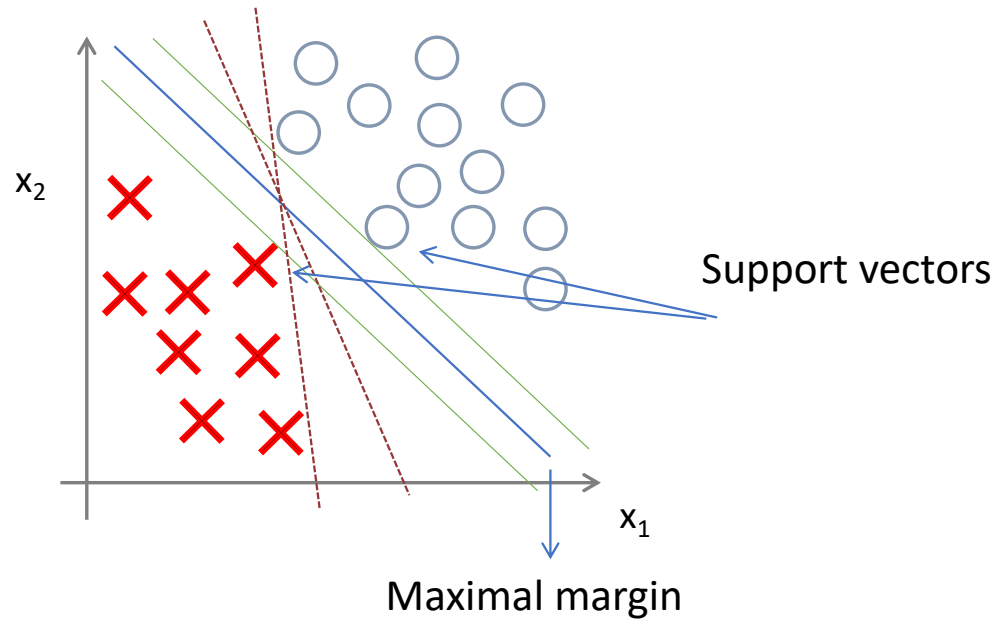
Eles são baseados em 2 ideias:

1 - usar apenas os exemplos relevantes no conjunto de treinamento (vetores de suporte) e

2 - utilizar uma transformação não linear aplicada às entradas para um espaço linear de linear através de uma função de kernel

(olha se para os pontos da fronteira e separa se)

Maximal margin classifier



Kernels

Define mappings, possibly nonlinear, to the original variables (features) by creating "new" features

Given original variables, calculate new features using similarity functions with predefined points

In SVMs, these predefined points are the examples themselves

Example: gaussian kernel

$$\begin{aligned} f_i &= \text{similarity}(x, l^{(i)}) \\ &= \exp \left(-\frac{\|x - l^{(i)}\|^2}{2\sigma^2} \right) \end{aligned}$$

(kernel gaussiano
os kernels são
definidos a partir
dos próprios dados
e depois aplica-se
a equação
gaussiana)

Defina mapeamentos, possivelmente não lineares, para as variáveis originais (recursos) criando "novos" recursos

Dadas as variáveis originais, calcule novos recursos usando funções de similaridade com pontos predefinidos

Em SVMs, esses pontos predefinidos são os próprios exemplos

SVMs in classification

The SMO training algorithm ensures that the optimal hyperplane of class separation is always achieved.

There are several types of kernels (linear, polynomial, Gaussian, splines), with Gaussian (RBF) being the most popular.

Two configuration parameters: $C > 0$ (a gap constant) and $\gamma > 0$ (the Gaussian kernel parameter).

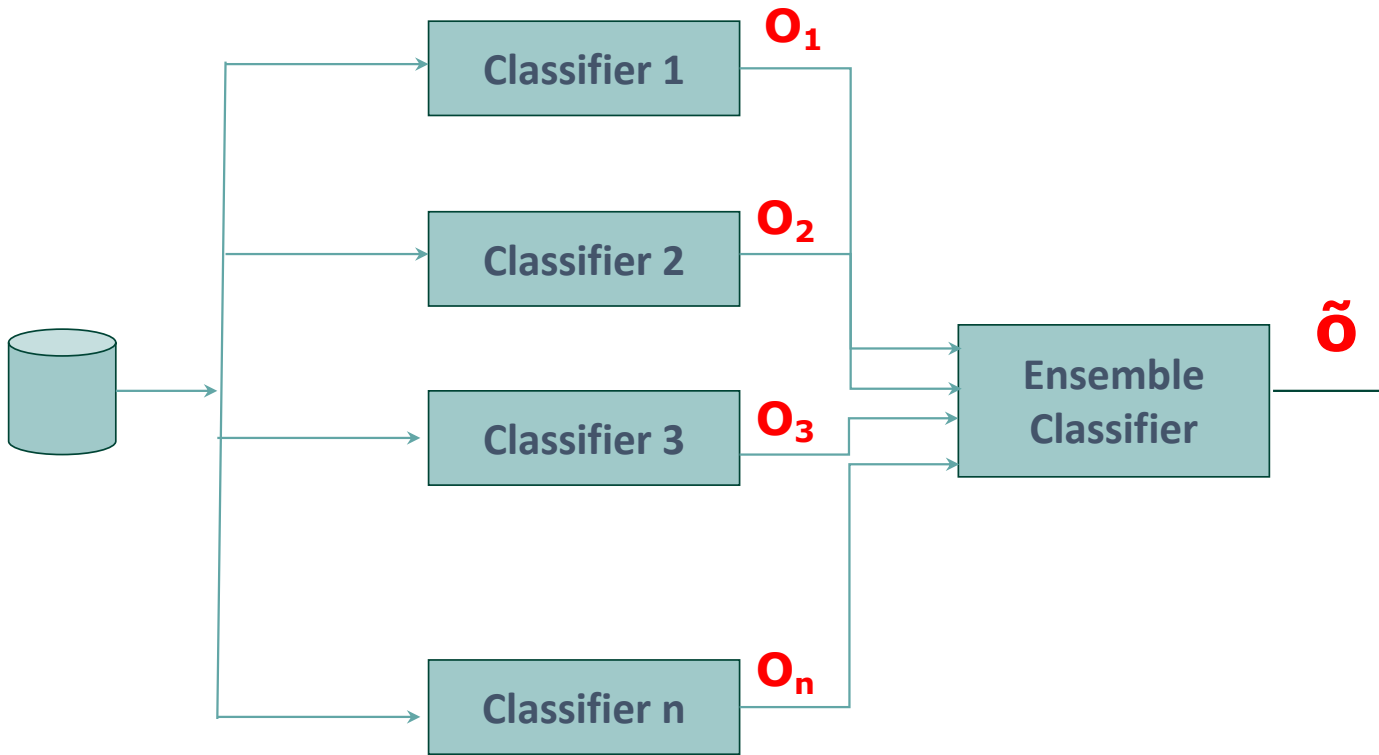
O algoritmo de treinamento SMO garante que o hiperplano ideal de separação de classes seja sempre alcançado.

Existem vários tipos de kernels (linear, polinomial, gaussiano, splines), sendo o gaussiano (RBF) o mais popular.

Dois parâmetros de configuração: $C > 0$ (uma constante de intervalo) e $\gamma > 0$ (o parâmetro do kernel gaussiano).

(os ensembles funcionam tanto melhor quanto mais classifiers porque ha mais diversidade)

Model ensembles



$$\tilde{O} = f(w_1 O_1, w_2 O_2, w_3 O_3, \dots, w_n O_n)$$

(nao torna as coisas mais simples. é preciso um set de L models, treina-los em dados diferentes pois assim vai falhar em sets diferentes e depois combina-los)

Model ensembles

Idea: response to new situations obtained from the combination of the responses of several different models

Components:

Set of L models distinct (classification or regression)

Combination function c which, given the outputs of the models, returns a single output value

Tasks:

How to build the model ensemble?

Which combination function to use?

Ideia: resposta a novas situações obtidas a partir da combinação das respostas de vários modelos diferentes

Componentes:

Conjunto de modelos L distintos (classificação ou regressão)

Função de combinação c que, dadas as saídas dos modelos, retorna um único valor de saída

Tarefas:

Como construir o conjunto modelo?

Qual função de combinação usar?

Construction of model ensembles

Manipulating the **training examples** in different ways to create different models:

- **Bagging**: each sample created by a distinct bootstrap process;
- **Boosting**:
 - Each example will have a probability of being chosen;
 - The choice is made considering the probabilities;
 - After creating each set of examples, probabilities are updated by decreasing the probabilities of the correctly ranked examples and increasing the rest.

Manipular os exemplos de treinamento de diferentes maneiras para criar diferentes modelos:

- Bagging: cada amostra criada por um processo de bootstrap distinto;
- Boosting:
 - Cada exemplo terá uma probabilidade de ser escolhido;
 - A escolha é feita considerando as probabilidades;
 - Depois de criar cada conjunto de exemplos, as probabilidades são atualizadas diminuindo as probabilidades dos exemplos classificados corretamente e aumentando o restante.

(vê-se quais os modelos que esta a prever bem e quais é que está a prever mal são conjuntos de árvores: cada árvore é treinada num diferente dataset cada árvore pode ser criada com diferentes atributos classificar com base numa votação entre as várias árvores)

Injetando aleatoriedade no algoritmo, mudando determinístico para decisões estocásticas

por exemplo, escolha o atributo para testar na raiz de uma árvore ou na totalidade dos atributos para testar em toda a árvore

Construction of model ensembles

Injecting randomness in the algorithm, changing deterministic to stochastic decisions

e.g choose attribute to test from the root of a tree or from the totality of attributes to test throughout the tree

...

Example: **Random forests**

- Ensemble of decision trees
- Random choice of attribute set to test
- Using bagging as a way to choose examples
- Tree induction algorithm: CART
- Uses the out-of-bag error (in the examples not chosen by bagging in each iteration) as a way to avoid overfitting

Exemplo: florestas aleatórias

- Conjunto de árvores de decisão
- Escolha aleatória do conjunto de atributos para teste
- Usar o bagging como forma de escolher exemplos
- Algoritmo de indução de árvore: CART
- Utiliza o erro out-of-bag (nos exemplos não escolhidos pelo bagging em cada iteração) como forma de evitar overfitting

(cria-se o modelo, prevê-se o y e há um erro associado ao y ($y_0 = \hat{y}_1 + e_1$)

$$\begin{array}{c} \hat{y}_2 + e_2 \\ | \\ \hat{y}_3 + e_3 \end{array}$$

$$y = (\hat{y}_1 + \hat{y}_2 + \hat{y}_3 + \dots) \times \text{shrinkage}$$

Construction of model ensembles

Gradient boosting

- Ensemble of simple models
- In each iteration we try to create a model F_{m+1} which is trained from the residues (errors) of F_m i.e. trained to learn the errors: $y - F_m(x)$
- Prediction values are given by the sum of the predictions of the various models
- It can be applied to various types of models, being quite popular with decision trees
- Normally used with "shrinkage", i.e. a multiplicative factor that reduces the contributions of each model to avoid over-adjustment

Aumento de gradiente

- Conjunto de modelos simples
- Em cada iteração tentamos criar um modelo F_{m+1} que é treinado a partir dos resíduos (erros) de F_m , ou seja, treinados para aprender os erros: $y - F_m(x)$
- Os valores de previsão são dados pela soma das previsões dos vários modelos
- Pode ser aplicado em diversos tipos de modelos, sendo bastante popular com árvores de decisão
- Normalmente usado com "encolhimento", ou seja, um fator multiplicativo que reduz as contribuições de cada modelo para evitar ajustes excessivos

Combination functions

Classification:

- **Voting**: select class that gathers the most "votes"
- **Confidence**: output is decided by the model that presents greater confidence associated with the class it proposes (or a weighted voting is done)

Regression:

- **Mean**: output is the average of the outputs of the models
- **Weighted mean**: weights of each model can be calculated based on a previous error estimation process, e.g. with validation examples

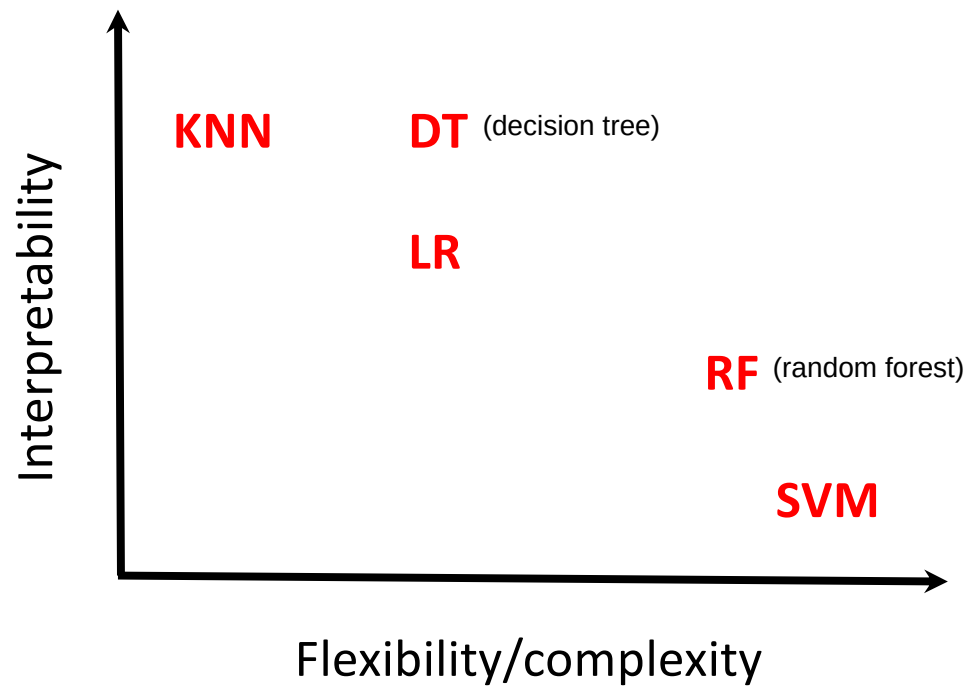
Classificação:

- Votação: selecione a turma que reúne mais “votos”
- Confiança: o resultado é decidido pelo modelo que apresenta maior confiança associada à classe que propõe (ou é feita uma votação ponderada)

Regressão:

- Média: o resultado é a média dos resultados dos modelos
- Média ponderada: os pesos de cada modelo podem ser calculados com base em um processo anterior de estimativa de erro, por exemplo. com exemplos de validação

Classification model comparison



(quanto mais complexo mais difícil de interpretar)

ex: interpretar uma decision tree é mais fácil do que interpretar uma random forest)

Implementing supervised ML pipelines in Python



- **Scikit-learn** includes several models and algorithms for **supervised** machine learning, including classification and regression, ensembles, error metrics and error estimation, feature selection, hyper-parameter optimization, among others
- Supervised models for classification and regression are implemented in **classes** that follow a single interface, which allows to easily test and compare different models in the same pipelines, even including ensemble based models

• O Scikit-learn inclui vários modelos e algoritmos para aprendizado de máquina supervisionado, incluindo classificação e regressão, conjuntos, métricas de erro e estimativa de erro, seleção de recursos, otimização de hiperparâmetros, entre outros

• Modelos supervisionados para classificação e regressão são implementados em classes que seguem uma única interface, o que permite testar e comparar facilmente diferentes modelos nos mesmos pipelines, incluindo até mesmo modelos baseados em ensemble

Implementing supervised ML pipelines in Python: models



Available classes implementing models for classification and regression include:

- Linear models – linear and logistic regression (with and without regularization), PLS
 - SVMs
 - KNN
 - Naïve Bayes
 - Decision trees
 - Ensemble models: bagging, boosting, voting classifiers, ...
 - etc
- As classes disponíveis que implementam modelos para classificação e regressão incluem:
- Modelos lineares – regressão linear e logística (com e sem regularização), PLS
 - SVMs
 - KNN
 - Baías ingênuas
 - Árvores de decisão
 - Modelos de conjunto: bagging, boosting, classificadores de votação, ...
 - etc.

https://scikit-learn.org/stable/supervised_learning.html#supervised-learning

Implementing supervised ML pipelines in Python: building, training, using models



The main functions included in all these are:

- Constructors – specific for each class implementing different classifiers, allowing to specify the desired parameters
- Function *fit* – allows to train a model given a training set (with known inputs and outputs)
- Function *predict* – allows to predict the output for a set of given inputs
- Function *score* – calculates the default scoring metric for the model, given a set of inputs (é preciso ter cuidado com o score porque pode usar diferentes métricas de erro)
- Specific functions are also available for the different error metrics (accuracy, precision, F1, R2, MSE, MAE, etc)

As principais funções incluídas em tudo isso são:

- Construtores – específicos para cada classe implementando diferentes classificadores, permitindo especificar os parâmetros desejados
- Função Fit – permite treinar um modelo dado um conjunto de treinamento (com entradas e saídas conhecidas)
- Função Predict – permite prever a saída para um conjunto de determinadas entradas
- Função Score – calcula a métrica de pontuação padrão para o modelo, dada um conjunto de entradas
- Funções específicas também estão disponíveis para as diferentes métricas de erro (exatidão, precisão, F1, R2, MSE, MAE, etc)

Implementing supervised ML pipelines in Python: error estimation



Regarding error estimation, there are functions to support the different methods

- Function *train_test_split*: can be used to split the data into training and test sets
- Function *cross_val_score*: can be used to run cross validation processes over a dataset
- There are several classes that allow to configure more elaborate error estimation methods (e.g. LOO, resampling, etc)

https://scikit-learn.org/stable/modules/cross_validation.html

Em relação à estimativa de erros, existem funções para apoiar os diferentes métodos

- Função *train_test_split*: pode ser usada para dividir os dados em conjuntos de treinamento e teste
- Função *cross_val_score*: pode ser usada para executar processos de validação cruzada em um conjunto de dados
- Existem diversas classes que permitem configurar métodos de estimativa de erro mais elaborados (por exemplo, LOO, reamostragem, etc)